

基于时间序列预测的蔬菜进货与定价决策模型

罗浩轩 黄路阳 杨镇泽

2026 年 3 月

摘要

针对生鲜商超蔬菜类商品易损耗、短保质期和销售空间有限等特点，本文围绕蔬菜商品预测数据后的每日进货决策与定价决策问题展开研究。

首先，在数据预处理阶段，完成包括 One-Hot 编码在内的数据整理；并进一步基于供应连续性与季节集中度的综合考量，将单品划分为常年性、季节性和时令性蔬菜；同时使用 IQR 法对利润率异常值进行筛查，并清洗滞销无关单品，为后续建模提供可靠数据基础。

对于问题一，本文从销量占比、时间分布与相互关系三个层面开展分析。数据分析得到蔬菜单品销量呈显著长尾分布，前 30% 的单品贡献约 90% 的总销量、不同品类内部各单品的分布结构不同；且各品类或单品在时间维度上具有明显差异。进一步地，采用控制总销量变量的偏相关性分析识别出部分类别间的替代与互补关系，并通过 FP-Growth 关联规则挖掘出若干强共现单品组合。

对于问题二，本文基于经济学中的双对数需求模型估计价格弹性，构建定价-销量修正函数，并求解得到各品类的需求价格弹性；同时采用 LSTM 模型预测未来 7 天各品类的销量与成本，求解后引入岭回归算法进行对比，说明 LSTM 在本场景下的良好适配，预测结果优秀。以上在此基础上建立收益最大化连续优化模型，并利用改进 GBest-PSO 算法求解，得到未来一周的最优补货与定价方案。

对于问题三，从第二问出发，本文考虑将新增“需求度”作为第二个优化目标纳入模型。并首先利用 VIKOR 法和熵权法确定单品需求度量化公式；而由于需要选择单品，构建了 01 决策变量来表征选择，同时增添了其他额外限制，建立双目标混合优化模型。通过引入 BG-RI 混合编码、先验知识和全局存档机制的改进 NSGA-II 算法求解，并最后基于膝点法从 Pareto 前沿中选取最优方案，实现了收益与需求满足的平衡性决策。

关键词：商超进货；偏相关性分析；FP-Growth 关联分析；LSTM 时间序列预测；双对数需求模型；GBest-PSO；VIKOR 法；NSGA-II

1 问题重述

1.1 问题背景

在生鲜商超经营中，多数蔬菜由于保鲜需求高，会随销售时间推移而逐步下降，故需要在每日凌晨依据历史销售与供应信息提前完成补货决策。而该决策是没有得到当日的销售数据和批发价格数据的，因此只能基于历史数据进行预测与判断。

与此同时，蔬菜商品的售价通常采用成本加成定价方法确定，加成率的决策也会影响销量；对运输损耗较大的商品，则会以折扣方式处理销售。由于蔬菜销量在需求侧与时间因素密切相关，在供给侧呈现出不同月份丰富的品类和单品不同的特征。故如何在多重约束下制定合理的补货与定价策略、实现收益最大化，成为商超经营中的关键问题。

1.2 待求解问题

结合题意，本文拟求解以下三个问题：

问题一：通过整理和处理后的历史数据，分析蔬菜各品类及单品销售量的分布规律与相互关系，以辅助后续进货决策；

问题二：通过时间序列预测模型预测 2023 年 7 月 1 日至 7 月 7 日的各品类销售量，并分析成本加成定价对销量的影响，构建模型计算各品类的进货策略与定价策略，使商超收益最大；

问题三：在添加新的约束条件后，将“需求度”纳入考量，重构优化模型，并求解得到 7 月 1 日的进货策略和定价策略。

2 模型假设

1. 假设短期内商超的进货与售价不会对顾客忠诚度造成显著影响，即顾客不会因为一周内的价格调整而立即改变长期消费习惯；
2. 假设该商超在过去三年中的补货与定价策略已在一定程度上反映了顾客消费需求，因此历史销售数据能够作为未来短期预测和优化决策的有效依据；
3. 假设该商超附近不存在显著影响销量波动的恶性竞争对手，从而不考虑由竞争性降价、抢占客流等行为引起的异常销售波动；
4. 假设蔬菜的品相损耗率在短期内保持相对稳定，因此可将附件 4 中给出的近期损耗率视为预测期内的损耗水平代表值。

3 问题分析

针对本题后续的分布分析、预测模型和优化模型，有必要首先对于已有数据进行预处理。在完成了表格间的自然连接后，我们首先对时间变量做了格式转化，将分类数据转化为独热编码，方便后续模型训练；之后，依照时间维度，将数据聚合到每日，方便后续以天为单位进行计算。基于该聚合数据，本研究发现不同单品的供应时间存在显著差异，进而提出了基于供应连续性与季节集中度的双指标分类方法，将单品划分为常年性、季节性和时令性三类。

此外，还需要剔除数据集中的异常值和无关值。

3.1 问题一的分析

“分布规律”方面，本文拟从以下两个方面展开：一是销量占比结构，即分析不同品类和不同单品的销售贡献程度，从而识别商超销售的核心品种或是构成结构；二是时间维度上的分布规律，即考察不同品类或是与处理中的时间单品分类在在时间上的销售规律。

“相互关系”方面，考虑到生鲜销量容易受到节假日、季节变化、总体消费水平等宏观共同因素的干扰，进而产生伪相关，故采用偏相关性分析品类间的关系。

而对于单品之间的关系，由于单品数量较多，计算量极大。考虑到单品间关系更可能表现为“共现”而非稳定线性相关，因此采用关联规则挖掘方法，从日交易或日上架集合中发现频繁组合及强关联模式。

3.2 问题二的分析

问题二要求从品类层面给出未来一周的进货策略与定价策略，这涉及销售和成本预估、定价影响和优化模型：

首先，建立定价与销量之间的关系模型。考虑到实际销量很受定价策略影响，引入经济学中的双对数需求模型，利用 SPSSPRO 平台得到各品类的需求价格弹性。

其次，预测未来七天的基础销量与成本。由于蔬菜销量和批发价格均为典型时间序列数据，可能具有趋势、周期和随机波动特征，传统线性模型难以充分刻画其非线性时序依赖关系。因此，本文将同时测试岭回归模型和 LSTM 模型的拟合程度，采用更好的 LSTM 模型来得到未来七天在“基准定价”条件下的预测值，并考虑纳入一传统时间序列模型进行对比。

最后，建立并求解优化模型。以利润最大化为目标，以销量不超过补货量、加成率处于合理区间等为约束，建立连续型非线性规划模型。并基于模型的数据规模和非线性特征，选取合适的算法进行求解。

3.3 问题三的分析

与问题二相比，本文首先将研究对象转向了单品，这会带来更高的计算量；其次，引入了销售端的需求，需要使用恰当定义量化需求度，并在优化模型中尽可能需求最大化，如作为第二个优化目标；同时，并不是所有单品都要选取，即需要引入 0-1 决策变量，重新构建混合数据类型的模型。

首先，可以对于参与计算的单品进行筛选以简化计算，该过程也包含了需求度的量化方法和排序。此处也需要合理选取适合这个数据量和场景特征的排序方法。

对于这个新的混合类型优化模型，还需要增加销售空间限制以及最小进货量限制等约束条件。同时考虑建立多目标优化模型，将已经量化的需求度作为第二个目标函数。而两个指标之间显然存在权衡，即选择更多高需求但低利润的单品会提高满足度，却可能压缩整体收益；反之，只保留高利润单品则会牺牲顾客需求覆盖率。

考虑使用经典的 NSGA-II 多目标遗传算法进行求解，得到一组 Pareto 非支配解，并设法从中选取一个兼顾利润与需求满足的最终实施方案。

4 符号说明

表 1: 主要符号及其含义

符号	意义	符号	意义
Q_i	品类 i 的总销量	Q_{ij}	品类 i 中单品 j 的销量
P_i	品类 i 的平均定价	P_{ij}	品类 i 中单品 j 的定价
L_i	品类 i 的平均损耗	L_{ij}	品类 i 中单品 j 的损耗
B_i	品类 i 的总补货量	B_{ij}	品类 i 中单品 j 的补货量
C_i	品类 i 的补货成本	C_{ij}	品类 i 中单品 j 的成本
w_i	品类 i 的平均加成率	w_{ij}	品类 i 中单品 j 的加成率
M	总销售额	T	总成本
k	损耗折扣率	D_j	需求度
E_p	需求价格弹性	E_c	交叉弹性
E_v	价格预期弹性		

5 数据预处理

概览题设所给出的附件信息，可以发现其数据量庞大、信息冗杂琐碎，为保证后续模型建立与求解的准确性，本文首先对原始数据进行系统性的整合、分析与清洗。

5.1 数据整合

5.1.1 整理关联

附件一与附件四共同描述了单品的编号、分类、近期损耗率信息，附件二与附件三共同描述了单品的销售、进购信息。为获得包含完整字段的统一数据集，本文通过 Python 编程以共同主键，将分散在多个附件中的信息整合至统一的宽表结构中，便于后续统一查询与计算。

5.1.2 变量处理

原始数据中存在类别型变量与时间型变量，需要进行针对性转换。

(1) 时间序列处理。将附件中的“销售日期”与“扫码销售时间”两列合并处理为标准的 `datetime` 时间序列格式，以便按不同时间粒度进行聚合与周期性分析。

(2) **One-hot** 编码处理。原始数据中的“销售类型”与“是否打折销售”均为类别型变量。本文采用 One-hot 编码将其转换为数值型分类变量。

One-hot 编码的基本思路是将分类转化为高维向量，防止数字或字母分类在模型训练时引入虚假大小关系。同时向量数据类型兼容性良好，适合直接输入到机器学习模型中进行训练与预测。

5.1.3 数据聚合

由于蔬菜单品的销售信息以逐笔交易的形式记录，信息粒度过细，因此本文将其以单品和品类两种维度，分别按日、周、月、年四种时间粒度进行数据聚合。以品类的日聚合为例，介绍聚合规则如下。

(a) 日销售量聚合。品类 i 在某日的总销量 Q_i 为该品类下所有单品 j 当日销量之和：

$$Q_i = \sum_j Q_{ij} \quad (1)$$

其中 Q_{ij} 表示品类 i 中单品 j 的当日销量。

(b) 日加权平均指标。对于日销售单价、进价、加成率等指标 x ，品类 i 的日聚合值 X_i 采用以销量为权重的加权平均：

$$X_i = \frac{\sum_j Q_{ij} \times x_{ij}}{Q_i} \quad (2)$$

其含义可以理解为以各单品销量为权重对该指标进行加权求均值。其中，加成率（利润率）的计算公式为：

$$\text{加成率} = \frac{\text{定价} - \text{成本}}{\text{成本}} \quad (3)$$

5.2 初步数据分析

5.2.1 单品按时间供应分类

(1) 按时间特性分类的必要性

对蔬菜的进货信息进行初步观察，发现不同单品的供应季节、周期和持续时间存在显著差异。查阅相关资料并结合题目信息，本文从供应时间的角度将蔬菜单品分为以下三类：

- 常年性蔬菜：可以在不同的地区生产，几乎整年都可以获得，供应高度连续。
- 季节性蔬菜：受气候和地理条件影响，仅在特定的季节内集中供应与销售。
- 时令性蔬菜：仅在某些特定的节日、假期、节气等短期时段出现，供应天数极少。

而这三类蔬菜在销量预测、需求弹性估计和补货决策中的行为模式存在本质不同，因此有必要在数据分析阶段对其进行系统分类，以便后续针对不同类型单品可以采用差异化的分析方法和模型构建策略。

(2) 改进的双指标分类方法 为了综合、合理地对所有单品进行分类，我们考虑了供应连续性和季节集中度两个方面，并希望通过一种同时考虑这两指标的分类方法来实现科学的划分。

指标一：供应连续性指标 S_c ，衡量单品在整个数据跨度中的供应频率：

$$S_c = \frac{\text{三年总供应天数}}{\text{数据跨度总天数}} \quad (4)$$

其中数据跨度总天数为 2020 年 7 月 1 日至 2023 年 6 月 30 日，共计 1095 天。 S_c 越接近 1 表明供应越连续。

指标二：季节集中度指标 S_s ，衡量单品供应在各季节中的集中程度：

$$S_s = \frac{\max_{k \in \{\text{春, 夏, 秋, 冬}\}} N_k}{\sum_k N_k} \quad (5)$$

其中 N_k 为该单品在季节 k 的总供应天数。 S_s 越接近 1 表明供应越集中在某一特定季节。

基于上述双指标，我们不断调整后设定如下分类规则：

1. 常年性蔬菜： $S_c > 0.8$ ，即三年中平均每年供应天数大于约 292 天，全年持续供应。
2. 时令性蔬菜： $S_c < 0.05$ ，即三年总供应天数不足约 55 天，属于短暂出现的特殊品种。
3. 季节性蔬菜： $0.05 \leq S_c \leq 0.8$ 的单品初步归为季节性蔬菜。进一步通过 S_s 进行验证，若 $S_s < 0.6$ 且对应季节 $k^* \neq \arg \max_k N_k$ ，则判定不为典型的季节蔬菜，根据其 S_c 的值与 0.8 的接近程度重新判断为常年性或是时令性蔬菜。

(3) 分类结果与验证。经过上述分类规则处理，251 种蔬菜单品的分类结果如下表所示。

表 2: 各类型蔬菜分类结果

蔬菜类型	占比	举例
常年性	12%	大白菜、胡萝卜
季节性	60.6%	莲藕、蒲公英
时令性	27.5%	槐花、藕

为验证分类的准确性，本文进一步根据各季节性蔬菜在进货信息中常出现的月份，推测其所属季节，并与生活常识进行比对。例如，“四川红香椿”经本文方法被判定为春季蔬菜 ($S_s = 0.87$ ，供应集中在 3–4 月)，查阅资料后发现其 3 月末到 4 月初确为采摘和食用的黄金时间，验证了分类的准确性。

5.3 加成率异常数据处理

将附件二与附件三自然连接之后，可以计算每条销售记录的加成率，进而发现其中存在不易被直接观察到的异常数据。大部分单品的加成率集中不超过 1，然而存在部分记录的加成率极高，甚至有的高达 200 以上，显然不符合蔬菜一般定价规律。若不加以处理，这些极端值将严重影响后续双对数需求模型的参数估计和价格弹性分析的准确性。本文考虑采用四分位距法对异常加成率进行清洗。

考虑到实际利润率数据的分布可能存在偏态，一般的 3σ 法则对非正态分布数据的检测效果可能不够理想。为此，本文引入不依赖分布假设的 IQR 方法作为补充。其优势在于，其仅基于数据的分位数统计量，对偏态分布的利润率数据具有更好的鲁棒性。具体步骤如下：

1. 计算利润率数据的第 25 百分位数 Q_1 和第 75 百分位数 Q_3 ；
2. 计算四分位距 $IQR = Q_3 - Q_1$ ；
3. 判定异常值：

$$x < Q_1 - 1.5 \times IQR \quad \text{或} \quad x > Q_3 + 1.5 \times IQR \quad \implies \quad x \text{ 为异常值} \quad (6)$$

经过上述处理后，商品加成率分布如下图所示，加成率主要集中在 $[0, 1]$ 区间内，分布形态合理，符合蔬菜类商品的一般定价特征。

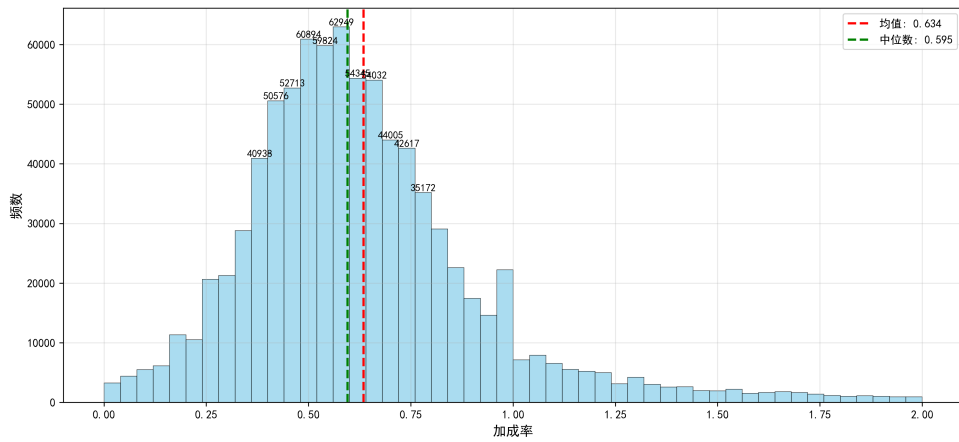


图 1: 异常值处理后的商品加成率分布

5.4 无关数据处理

观察蔬菜销售数据发现，部分蔬菜在供应上未呈现周期规律性，且供应天数极少、销售量同样极少。本文认为该类蔬菜可能为某段时间突然出现的特殊品种，存在供应链不稳定或市场需求量极少的情况，不具备稳定的销售与供应规律。在数据分析中，这类数据属于无关数据，对蔬菜总销量的规律分析以及未来预测均无有利贡献，反而可能引入噪声干扰模型的训练与估计。因此，本文不予将其纳入后续分析。

具体处理规则如下：

- 规则一：剔除完全没有销量数据的蔬菜单品，共计 5 个。这些单品在附件一中有登记，但在附件二的销售流水中从未出现过。
- 规则二：剔除销售天数小于 10 天、销售量占总销量比重小于 3‰、且无年周期规律的蔬菜单品，共计 48 个。这些单品的供应不稳定、销量极低，不具备分析价值。

经过上述无关数据处理后，剩余 198 种蔬菜单品用于后续的分析与建模。

6 品类或单品分布及相关性分析

6.1 销量分布规律的探究

6.1.1 各单品总销量的长尾分布特征

将数据预处理后保留的 198 种蔬菜单品按三年总销量从高到低排序，绘出各单品总销量的直方图以及累计销量占比曲线，如下图所示。

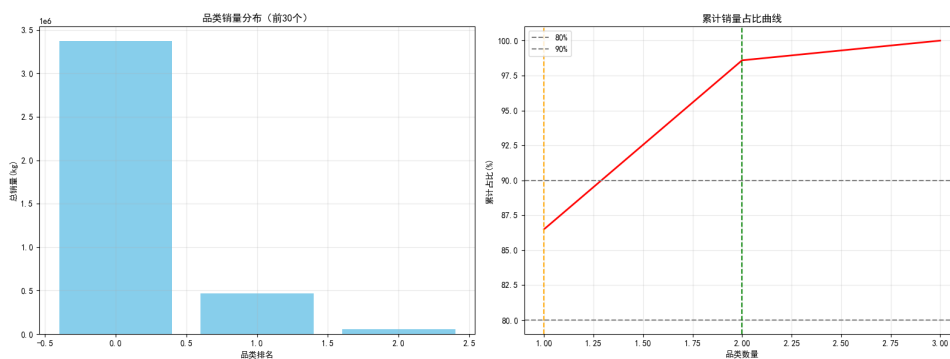


图 2: 各单品总销量的分布

分析上图可以发现。单品销量呈现显著的长尾分布。

少数单品贡献了绝大部分销量，而大部分单品的销量相对较低。具体而言，前 30% 的蔬菜单品总销量贡献率达到约 90%，呈现出典型的“二八法则”特征。这说明商超蔬菜销售高度依赖少数畅销品种，对补货决策有重要指导意义——应重点保障高销量单品的供给。

6.1.2 品类内部单品销量差异分析

进一步观察各品类内部单品的销量分布，发现不同品类呈现出差异化的内部结构。以茄类和食用菌类为代表进行对比分析，茄类中“紫茄子 (2)”一种单品即占据了 61% 的品类销量，呈现高度集中的特征；而食用菌类各单品销量分布则相对均匀，排名前三的单品占比均在 15%–20% 之间。各品类内部的不同差异化结构为问题三中单品的筛选与补货量分配提供了参考。

6.2 时间维度上的分布规律

6.2.1 各品类销量的时间分布规律

以月为最小时间单位，绘出 2020 年 7 月至 2023 年 7 月期间六个品类月销量的变化曲线，如图 3 所示，读图可以分析：

(1) 各品类在相同时间节点出现同步的尖峰或低谷。这说明各品类的销量受到共同外部因素（如宏观经济环境、季节更替、重大节日等）的影响。例如，每年春节前后各品类均出现明显的销量高峰，这与年货集中采购的消费习惯一致。

(2) 花叶类的波动幅度最大，且与总销量曲线相似度最高。这是因为花叶类不仅包含小白菜、小青菜等日常食用蔬菜（需求弹性低，属于生活刚需），还包含白蒿、鱼腥草等药用单品以及洪山菜薹珍品手提袋、洪山菜薹莲藕拼装礼盒等礼品类单品（需求弹性高，消费随外部因素波动大）。正是这种多元化的单品构成，使得花叶类品类整体波动剧烈。

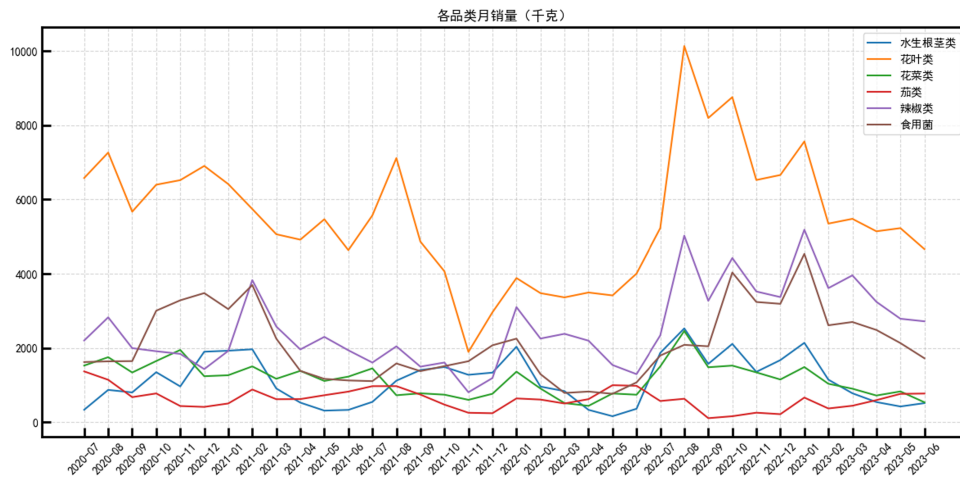


图 3: 各品类月销量的变化曲线

（3）茄类销量较为平稳，在较低销量水平上波动不明显。茄类单品种类少且均为日常食用品种，需求弹性较低，消费相对稳定。

6.2.2 各类型单品销量的时间分布规律

基于蔬菜分类结果，分别计算常年性、季节性、时令性三类蔬菜在每月的总销量，绘制月度销量变化曲线如下图所示。

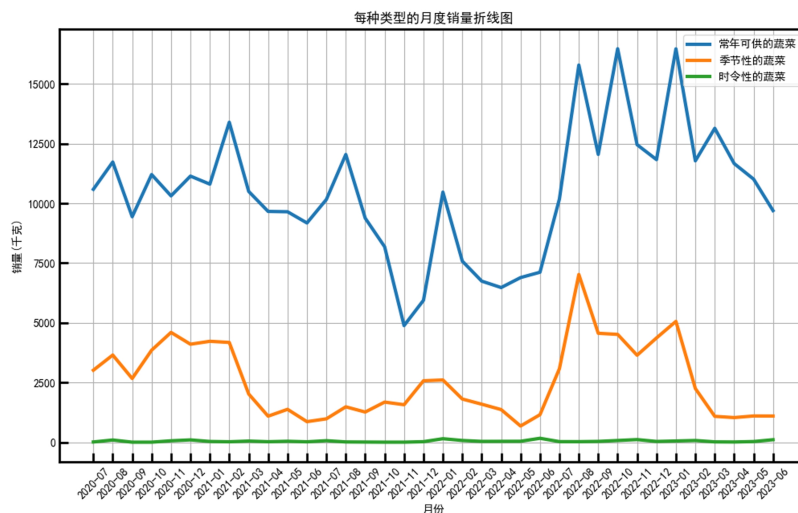


图 4: 各分类月销量的变化曲线

从图可分析归纳出如下规律：

1. 常年性蔬菜销量最高，其变化趋势与总销量极为相似。由于常年性蔬菜全年持续供应且包含大量日常消费品种，其销量在总销量中占据主导地位，因此其波动方向和幅度基本决定了总销量的变化态势。
2. 季节性蔬菜在不同季节中均有不同单品贡献销量，因此在全年范围内整体呈现出与总销量相似的变化状态。但从单品层面看，各单品仅在自身供应的季节存在销量分布。
3. 时令性蔬菜往往出现时间短暂，每种单品的销量仅分布在一年中的某几天，整体综合销量较低，随时间变化不明显，几乎在零值附近波动。这类蔬菜对总销量和品类销量的影响可以忽略。

6.3 相互关系的探究

6.3.1 偏相关性分析的必要性

探究品类销售量的相互关系时，一个关键的问题是不能直接使用简单的 Pearson 相关系数，这会带来不科学的结果。原因如下：

各品类的销售量数据均为时间序列，受到宏观经济环境、季节更替、疫情影响、重大节日以及消费者周内购物习惯等共同因素的影响。这些因素会导致所有品类的销量呈现“同涨同跌”的趋势，从而在简单相关分析中产生伪相关现象。例如，花叶类和茄类可能在春节期间同时出现销量高峰，直接计算相关系数会得到显著的正相关结果，但这并不意味着两者之间存在真实的替代或互补关系。

该现象在本场景中体现相当明显，即若直接计算各品类之间的 Pearson 相关系数，结果如下图。

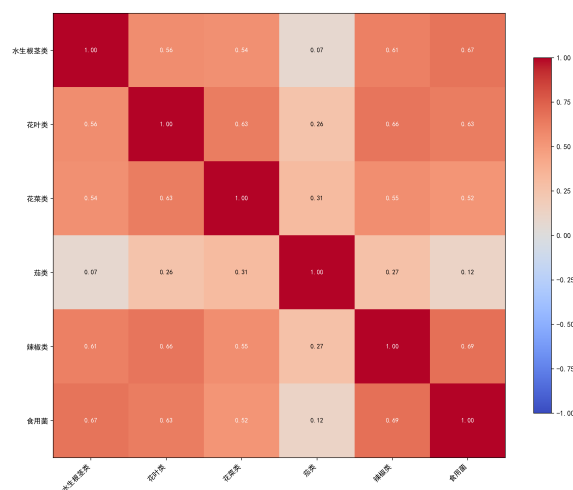


图 5: 各品类销售总量的 parson 相关性

可以发现几乎任意两品类之间的相关系数都存在较高的相关系数，这明显是由于共同外部因素导致的伪相关，而非品类间的内在关联。为解决这一问题，本文采用偏相关

性分析，通过控制第三方变量来剥离宏观共同因素的影响，揭示两个品类之间的净相关关系。

6.3.2 偏相关性分析模型建立

(1) 选择恒定控制变量

本文选取日总销售量作为控制变量 Z 。日总销售量是所有品类销量之和，能够综合反映宏观经济波动、季节性需求变化、重大节日等的共同影响。

通过控制总销售量 Z ，可以各因素对各品类销量的共同影响，从而识别品类间由消费者偏好、替代效应或互补效应驱动的内在关联。

(2) 偏相关系数的计算 设 X 、 Y 分别为两个品类的日销量序列， Z 为日总销售量。在控制 Z 的条件下， X 与 Y 的偏相关系数定义为：

$$r_{XY \cdot Z} = \frac{r_{XY} - r_{XZ} \cdot r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}} \quad (7)$$

其中 r_{XY} 、 r_{XZ} 、 r_{YZ} 分别为 X 与 Y 、 X 与 Z 、 Y 与 Z 之间的 Pearson 简单相关系数。

偏相关系数 $r_{XY \cdot Z}$ 的含义为：在去除 Z 的线性影响之后， X 与 Y 之间残余的线性相关程度。其取值范围为 $[-1, 1]$ ：正值表示在控制总销量后两品类仍呈正相关，负值表示呈负相关，接近零表示无显著关联。

(3) 假设检验

为判断偏相关系数是否显著，设定如下假设检验框架，以花叶类与茄类为例：

- 原假设 H_0 ：在控制日总销售量的条件下，花叶类与茄类不存在显著的偏相关性，即 $r_{XY \cdot Z} = 0$ ；
- 备择假设 H_1 ：在控制日总销售量的条件下，花叶类与茄类存在显著的偏相关性，即 $r_{XY \cdot Z} \neq 0$ 。

检验统计量为：

$$t = \frac{r_{XY \cdot Z} \sqrt{n - 2 - 1}}{\sqrt{1 - r_{XY \cdot Z}^2}} \quad (8)$$

其中 n 为样本量，即日数据天数。该统计量在 H_0 成立时近似服从自由度为 $n - 3$ 的 t 分布。当 p 值小于显著性水平 $\alpha = 0.05$ 时，拒绝 H_0 ，认为两品类间存在显著的偏相关性。

6.3.3 品类间偏相关性分析结果

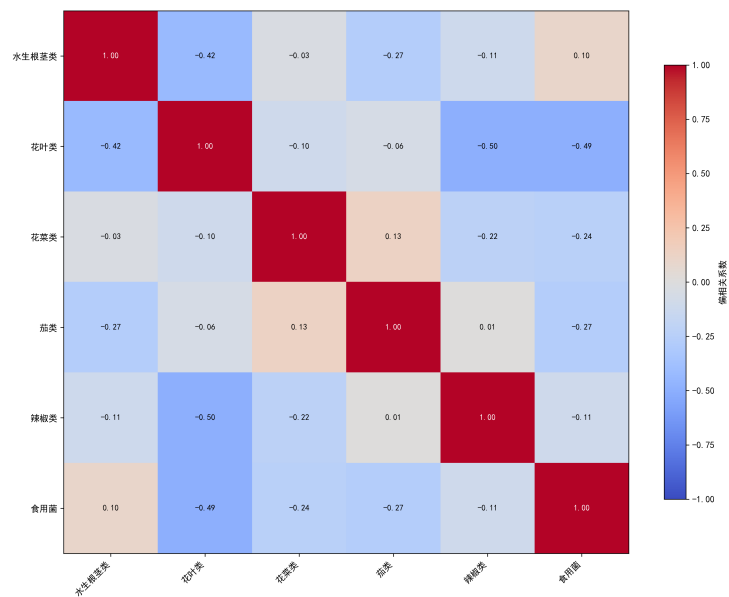


图 6: 各品类销售总量的偏相关性

分析上图可以得出以下主要发现：

- (1) 花叶类与水生根茎类、辣椒类、食用菌类存在相对较强的负偏相关性，且在 1% 显著性水平下显著。这说明在控制市场总体需求水平后，花叶类销量的提高会对水生根茎类、辣椒类、食用菌类的销量产生一定程度的挤占效应，花叶类与这些品类互为替代品。这可能是因为消费者在购买蔬菜时存在品类间的替代选择，即当花叶类品种丰富、价格适宜时，消费者可能减少对其他品类的购买，符合生活常识。
- (2) 其他品类之间的正负偏相关关系均不明显。这与蔬菜本身作为生活必需品的特征有关：消费者对蔬菜的总体需求相对稳定，品类间的替代与互补效应有限，主要受个人饮食偏好和当日供应情况的影响。这一结果与实际销售规律较为吻合。

6.3.4 单品间关联关系的分析方法选择

在单品层面探究相互关系时，本文未继续采用直接两两进行的偏相关性分析，而是选用关联规则挖掘方法，理由如下：

- 经过数据清洗后仍有约 198 种单品，若进行两两偏相关分析，需计算 $C_{198}^2 = 19503$ 对偏相关系数，计算量大且结果矩阵过于庞大，难以提取有价值的信息。
- 单品间的关系更多表现为共现关系，即某些单品经常在同一天被商超同时上架销售。假定商超在选定当日可售单品时已综合考虑了消费者的购买偏好和搭配需求，那么频繁共现的单品组合背后蕴含着重要的市场关联信息。

- 关联规则挖掘方法能够自动发现高频共现的单品组合，并以支持度和置信度量化关联强度，输出形式简洁易读。

6.3.5 FP-Growth 关联分析模型

在关联规则挖掘算法的选择上，本文考虑了经典的 Apriori 算法与 FP-Growth 算法，然而由于数据量过大，后者在效率上具有明显优势。FP-Growth 仅需两次扫描数据集，第一次统计各项频度，第二次构建压缩的 FP 树（Frequent Pattern Tree）数据结构。后续的频繁项集挖掘在 FP 树上递归完成，无需生成候选项集，在大规模数据集上效率远优于 Apriori，故最终选用。算法的具体步骤如下：

1. 构建项头表：扫描整个数据集，统计每个单品在所有交易日中出现的频度，将频度不低于最小支持度阈值的单品存入项头表，并按频度从高到低排序。
2. 构建 **FP** 树：对于每个交易日的单品集合，根据项头表中的顺序排列，依次插入 FP 树中。相同前缀的交易共享树中的路径节点，从而实现数据的高效压缩。
3. 构建条件模式基：对于项头表中的每个频繁项，收集其在 FP 树中所有出现路径的前缀，构成该项的条件模式基。
4. 递归挖掘频繁项集：对每个频繁项的条件模式基递归地构建条件 FP 树，并在条件 FP 树中继续挖掘频繁项集，直至条件 FP 树为空或仅含单一路径。
5. 生成关联规则：基于挖掘到的频繁项集，按照最小置信度阈值筛选生成关联规则，输出前项、后项、支持度和置信度。

6.3.6 单品间关联规则的求解结果

以每日出现的单品种类集合作为“事务”（transaction），设定最小支持度为 0.8，最小置信度为 0.9，通过 FP-Growth 算法挖掘单品间的关联规则。部分典型结果列举如表 3。

表 3: FP-Growth 部分关联规则求解结果

前项	后项	支持度	置信度
(102900005115823.0)	(102900005116257.0)	0.843	0.996
(102900005116714.0, 102900005115823.0)	(102900005116257.0)	0.837	0.996
(102900005116899.0, 102900005115823.0)	(102900005116257.0)	0.833	0.996

观察表上表中的关联规则可以发现：

- 上述关联规则的支持度均大于 **0.8**，说明相关单品组合在超过 80% 的交易日中同时出现，共现频率极高。
- 置信度均大于 **0.99**，说明当前项中的单品出现时，后项中的单品几乎必然同时出现，两者之间存在强关联关系。
- 这些强关联关系反映了商超在进货选品时的固定搭配模式，也间接反映了消费者对某些单品组合的稳定需求。

7 问题二的模型建立与求解

问题二要求以品类为单位，分析各蔬菜品类的销售总量与成本加成定价的关系，并给出未来一周（2023 年 7 月 1 日至 7 月 7 日）的日补货总量与定价策略，以最大化商超收益。

7.1 各蔬菜品类的销售总量与成本加成定价的关系

由于题中所给的三年销量为时间序列数据，其变化随时间、社会经济等因素变化程度十分明显，直接对销量与定价的关系进行相关性分析难以得出正确的结论。根据经济学理论，价格弹性指的是市场需求量随价格变动的变化程度，价格弹性可分为需求价格弹性、供给价格弹性、交叉价格弹性、预期价格弹性等类型。本题所要求 16 探究各品类蔬菜的销售量与其定价的关系，即为需求弹性

$$E_p = \frac{dQ/Q}{dP/P}$$

此处选用双对数需求模型对价格弹性进行估计。双对数需求函数是使用最为广泛的需求函数式，其最大的优点是能直接估计价格弹性，基本形式如下：

$$\ln Q_i = \alpha_i + \sum_k e_{ik} \ln P_k + e_i \ln x + \varepsilon_i$$

其中 q_i 表示每个品类的销售量， p_k 表示每个品类的平均销售价格， $x = \sum_i Q_i \times P_i$ 为总销售金额。 α_i 为常量参数， e_{ik} 为价格弹性即为 E_p ， e_i 为支出弹性。

7.1.1 Double-log 模型求解结果

利用 spsspro 平台采用最小二乘法 OLS 对双对数需求模型参数进行估计：

表 4: 双对数回归结果: 花叶销售量

变量	B	标准误	β	t	P	VIF
花菜价格 (对数)	0.443	0.557	—	0.796	0.428	—
花叶价格 (对数)	-0.173	0.091	-0.08	-1.908	0.059*	1.422
辣椒价格 (对数)	-0.984	0.129	-0.302	-7.643	0.000***	1.264
茄类价格 (对数)	-0.129	0.062	-0.098	-2.087	0.040**	1.799
水生根价格 (对数)	0.03	0.03	0.037	1.005	0.318	1.127
食用菌价格 (对数)	0.02	0.057	0.014	0.361	0.719	1.181
总支出 (对数)	0.924	0.038	0.921	24.482	0.000***	1.148

模型拟合指标: $R^2 = 0.886$, 调整 $R^2 = 0.878$, $F = 102.6^{***}$ ($P = 0.000$)。

注: * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$; 因变量为“花叶销售量 (对数)”。

表 5: 双对数回归结果: 花菜销售量

变量	B	标准误	β	t	P	VIF
花叶价格 (对数)	-0.429	0.251	-0.139	-1.709	0.091*	1.422
辣椒价格 (对数)	-0.584	0.355	-0.126	-1.643	0.104	1.264
茄类价格 (对数)	-0.073	0.170	-0.039	-0.428	0.670	1.799
水生根价格 (对数)	-0.047	0.157	-0.031	-0.301	0.764	2.292
食用菌价格 (对数)	-0.005	0.083	-0.005	-0.064	0.949	1.127
总支出 (对数)	1.069	0.104	0.748	10.252	0.000***	1.148

模型拟合指标: $R^2 = 0.573$, 调整 $R^2 = 0.540$, $F = 17.623^{***}$ ($P = 0.000$)。

注: * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$; 因变量为“花菜销售量 (对数)”。

表 6: 双对数回归结果: 辣椒销售量

变量	B	标准误	β	t	P	VIF
花菜价格 (对数)	0.105	0.910	—	0.115	0.909	—
花叶价格 (对数)	-0.090	0.148	-0.031	-0.603	0.548	1.422
辣椒价格 (对数)	-0.448	0.210	-0.102	-2.132	0.036**	1.264
茄类价格 (对数)	-1.044	0.101	-0.591	-10.371	0.000***	1.799
水生根价格 (对数)	0.138	0.049	0.126	2.799	0.006***	1.127
食用菌价格 (对数)	-0.048	0.092	-0.024	-0.516	0.607	1.181
总支出 (对数)	0.905	0.062	0.668	14.676	0.000***	1.148

模型拟合指标: $R^2 = 0.834$, 调整 $R^2 = 0.821$, $F = 65.949^{***}$ ($P = 0.000$)。

注: * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$; 因变量为“辣椒销售量 (对数)”。

表 7: 双对数回归结果: 茄类销售量

变量	B	标准误	β	t	P	VIF
花菜价格 (对数)	-0.034	0.285	-0.008	-0.118	0.906	1.422
花叶价格 (对数)	0.499	0.404	0.078	1.236	0.220	1.264
辣椒价格 (对数)	0.331	0.193	0.129	1.711	0.090*	1.799
茄类价格 (对数)	-0.392	0.178	-0.188	-2.203	0.030**	2.292
水生根价格 (对数)	0.588	0.094	0.373	6.238	0.000***	1.127
食用菌价格 (对数)	0.030	0.178	0.010	0.167	0.867	1.181
总支出 (对数)	1.111	0.118	0.566	9.385	0.000***	1.148

模型拟合指标: $R^2 = 0.708$, 调整 $R^2 = 0.686$, $F = 31.915^{***}$ ($P = 0.000$)。

注: * $p < 0.1$, ** $p < 0.05$, *** $p < 0.01$; 因变量为“茄类销售量 (对数)”。

表 8: 双对数回归结果：食用菌销售量

变量	B	标准误	β	t	P	VIF
花菜价格（对数）	0.037	0.200	0.010	0.185	0.853	1.422
花叶价格（对数）	0.233	0.283	0.044	0.824	0.412	1.264
辣椒价格（对数）	0.335	0.135	0.157	2.475	0.015**	1.799
茄类价格（对数）	-0.021	0.125	-0.012	-0.170	0.865	2.292
水生根价格（对数）	-0.227	0.066	-0.172	-3.440	0.001***	1.127
食用菌价格（对数）	-1.295	0.124	-0.534	-10.403	0.000***	1.181
总支出（对数）	0.968	0.083	0.589	11.654	0.000***	1.148

模型拟合指标： $R^2 = 0.795$ ，调整 $R^2 = 0.780$ ， $F = 51.001^{***}$ （ $P = 0.000$ ）。

注：* $p < 0.1$ ，** $p < 0.05$ ，*** $p < 0.01$ ；因变量为“食用菌销售量（对数）”。

表 9: 双对数回归结果：水生根销售量

变量	B	标准误	β	t	P	VIF
花菜价格（对数）	-1.284	0.577	-0.184	-2.224	0.029**	1.422
花叶价格（对数）	1.939	0.818	0.185	2.371	0.020**	1.264
辣椒价格（对数）	-0.413	0.392	-0.098	-1.053	0.295	1.799
茄类价格（对数）	0.185	0.361	0.054	0.513	0.609	2.292
水生根价格（对数）	-1.475	0.191	-0.568	-7.717	0.000***	1.127
食用菌价格（对数）	0.563	0.36	0.118	1.564	0.121	1.181
总支出（对数）	1.447	0.24	0.448	6.032	0.000***	1.148

模型拟合指标： $R^2 = 0.558$ ，调整 $R^2 = 0.524$ ， $F = 16.593^{***}$ （ $P = 0.000$ ）。

注：* $p < 0.1$ ，** $p < 0.05$ ，*** $p < 0.01$ ；因变量为“食用菌销售量（对数）”。

从花叶类的回归结果来看，模型的拟合优度 $R^2 = 0.886$ ，调整 $R^2 = 0.878$ ，拟合效果良好，而 $F = 102.6^{***}$ （ $P = 0.000$ ），对应 P 值小于 0.001 表明模型整体高度显著，所选价格变量及总销售额对花叶类销量具有显著的解释能力。各变量的 VIF 值均小于 2.3，远低于 10 的临界值，说明不存在多重共线性问题，系数估计结果稳健可靠。

7.2 未来 7 天相关数据确定

为了更好的对未来七天的定价以及补货情况进行决策，需要对未来七天的销量、品类平均成本、品类平均损耗率、损耗折扣进行确定。

7.2.1 损耗折扣与损耗率的确定

分析一天中同个单品打折与不打折的出售价格之比，进行加权平均，确定损耗折扣率 k 为 0.7。

$$k = \frac{\sum Q_{\text{打折}} \times P_{\text{折扣}}}{\sum Q_{\text{打折}}} \quad (9)$$

根据附件四中的单品损耗率对其加权平均，得到一种品类的损耗率 L_i ：

$$L_i = \frac{\sum_j Q_{ij} \times L_{ij}}{Q_i} \quad (10)$$

其中, Q_{ij} 表示品类 i 中单品 j 的销售量, L_{ij} 表示该单品的损耗率, $Q_i = \sum_j Q_{ij}$ 为品类 i 的总销售量。

7.2.2 时间序列预测

为了制定未来一周（2023 年 7 月 1 日至 7 月 7 日）的补货与定价策略, 需要提前获知各品类在接下来 7 天内的日销量和日批发成本。由于蔬菜销售数据具有明显的周期性（日、周、年周期）和趋势性, 普通回归模型难以准确预测。本文对比了两种预测方法: 岭回归和 LSTM 长短时记忆网络, 并最终选择预测精度更高的 LSTM 模型用于后续优化。

岭回归预测结果利用 spsspro 平台进行岭回归分析: 从岭回归的预测结果可以看出,

表 10: 各品类回归模型的拟合优度 R^2

品类	R^2
花菜类	0.304
花叶类	0.398
茄类	0.458
辣椒类	0.327
食用菌	0.326
水生根茎类	0.351

各品类销量预测的 R^2 普遍偏低（均在 0.3 ~ 0.46 之间）, 表明基于时间特征的线性模型难以准确刻画蔬菜销量的复杂波动规律。因此后续将采用 LSTM 长短时记忆网络模型进行预测。

LSTM 通过细胞状态和门控机制来管理控制信息的流动, 使其能够有效地捕捉时间序列数据中的长期依赖关系, 是一种强大的时间序列预测工具。主要优点可以概括为: 长期记忆能力: 引入“细胞状态”内部状态, 捕捉和保留序列中的长期依赖关系; 门控机制: 遗忘门、输入门和输出门这些门控单元通过学习来控制信息的流动, 有效地管理细胞状态的更新和选择性遗忘; 反向传播稳定性: 由于 LSTM 的门控机制, 它在反向传播时通常不会出现梯度消失或爆炸的问题, 训练更加容易。模型评价指标的选择由于均方误差、均方根误差、平均绝对误差的范围与数据本身大小密切相关无法直观的了解模型的效果, 因此本文主要考虑 R 方作为评价指标, 同时参考 MSE、RMSE、MAE 综合评估。LSTM 模型预测结果搭建 LSTM 模型对销售量及成本进行预测, LSTM 模型参数如下:

表 11: LSTM 模型参数设置

参数	取值
单层 LSTM 单元数	64
全连接层神经元数	1 （单步预测，输出未来 1 天）
迭代次数	400
损失函数	均方误差 (MSE)
优化器	Adam
样本批量	16
学习率	0.005
Dropout	0.3
回溯窗口 (look_back)	30 天
验证集比例	0.2
测试集天数	120 天

表 12: LSTM 与岭回归 R^2 对比

品类	岭回归 R^2	LSTM R^2
花菜类	0.304	0.5417
花叶类	0.398	0.5924
辣椒类	0.327	0.6207
茄类	0.458	0.6809
食用菌	0.326	0.5050
水生根茎类	0.351	0.6083

从上表可以看出，LSTM 模型效果要好于岭回归模型。这表明，成本数据具有较强的时序规律，LSTM 能够有效利用历史信息进行精准预测。下给出 LSTM 模型对未来 7 天成本及销量的预测值。

表 13: LSTM 销量预测结果汇总

品类	R^2	MSE	RMSE	MAE
花菜类	0.5417	83.52	7.32	5.76
花叶类	0.5924	1047.46	28.05	23.30
辣椒类	0.6207	396.67	18.27	10.25
茄类	0.6809	40.25	6.46	5.27
食用菌	0.5050	298.06	17.26	13.68
水生根茎类	0.6083	108.65	16.32	7.64

表 14: LSTM 未来 7 天销量预测值（2023 年 7 月 1 日—7 月 7 日）

品类	7 月 1 日	7 月 2 日	7 月 3 日	7 月 4 日	7 月 5 日	7 月 6 日	7 月 7 日
花菜类	27.30	28.00	29.71	29.66	26.72	28.39	30.01
花叶类	197.47	186.16	187.43	184.42	199.07	197.39	181.59
辣椒类	98.01	98.05	95.36	105.30	96.77	106.50	92.76
茄类	32.04	29.52	34.37	26.41	26.70	35.63	28.72
食用菌	54.04	59.79	66.26	57.32	58.05	65.89	52.13
水生根茎类	29.91	28.88	26.32	26.38	25.53	30.06	25.47

表 15: 完整成本预测结果（各品类评价指标）

品类	R^2	MSE	RMSE	MAE
花菜类	0.8477	0.18	0.43	0.31
花叶类	0.4942	0.57	0.76	0.49
辣椒类	0.8696	0.38	0.62	0.45
茄类	0.8094	0.15	0.38	0.28
食用菌	0.4766	0.70	0.83	0.39
水生根茎类	0.6615	1.59	1.26	0.61

表 16: 未来 7 天成本预测值（2023 年 7 月 1 日—7 月 7 日）

品类	7 月 1 日	7 月 2 日	7 月 3 日	7 月 4 日	7 月 5 日	7 月 6 日	7 月 7 日
花菜类	7.86	7.73	7.63	7.57	7.62	7.67	7.70
花叶类	3.90	3.82	3.34	3.52	3.67	3.78	3.50
辣椒类	5.78	5.83	5.78	5.67	5.73	5.72	5.74
茄类	4.44	4.47	4.47	4.59	4.71	4.76	4.85
食用菌	4.85	4.83	4.80	4.77	4.93	4.98	5.00
水生根茎类	11.89	12.14	12.43	12.28	11.92	11.71	11.67

7.3 商超补货与定价策略研究

由于蔬菜类商品的进货量与销售量仅受当天商超定价策略以及消费者需求量的影响，并且消费者需求同样仅受当日客观因素的影响，因此可以认为每日销售量与盈利均为独立变量，不会产生相互影响。基于此，可将对 7.1-至 7.7 的定价策略进行单独考虑，分别确定每日的定价与补货策略。

由 Double-log 模型回归结果可知定价对销售量会产生负相关性的影响，因此本文对 LSTM 预测出未来七天的销售量做一个修正，根据需求价格弹性的含义可知，价格变化率与销售量变化率的比值为需求价格弹性系数，因此本文参照双对数需求模型建立以下销量-定价函数：

$$\ln Q = \ln Q_{\text{预测值}} + E_p \ln \frac{P}{P_{\text{预测}}} \tag{7}$$

引入定价作为外生变量后，结合 LSTM 时间序列以求得的模型对真实销量进行拟合，发现加入变量定价后，模型拟合效果较优，说明本文建立的价格弹性修正函数与求得需求价格弹性参数可靠，各品类需求弹性价格参数如下所示：

表 17: 品类需求弹性参数

品类	花菜类	花叶类	辣椒类	茄类	食用菌类	水生根茎类
弹性系数	-1.92	-0.173	-0.448	-0.392	-1.295	-1.475

7.3.1 收益最大的优化模型建立

决策变量本问题需要确立六种品类的定价方案以及补货量，根据成本加成定价法，只需确定六种品类的加成功率即可确定最终售价，因此确定决策变量为：

- 加成率： w_i , ($1 \leq i \leq 6$)
- 补货量： B_i , ($1 \leq i \leq 6$)

因此某品类的定价为：

$$P_i = (1 + w_i) \times C_i \quad (11)$$

约束条件由于销量不可能大于进货量，因此确定约束条件：

$$Q_i \leq B_i \quad (12)$$

目标函数在此基础上计算该超商一天的销售金额 M ，以及总成本 T 为：

$$M = \sum_{i=1}^6 Q_i \times L_i \times P_i \times k + Q_i \times (1 - L_i) \times P_i \quad (13)$$

$$T = \sum_{i=1}^6 B_i \times C_i \quad (14)$$

为使商超利润最大，则目标函数为：

$$f = M - T \quad (15)$$

连续最优化模型

$$\max f = M - T \quad (16)$$

s.t.

$$\left\{ \begin{array}{l} B_i, w_i \quad (1 \leq i \leq 6) \\ P_i = (1 + w_i) \times C_i \\ \ln Q = \ln Q_{\text{预测值}} + E_p \ln \frac{P}{P_{\text{预测}}} \\ Q_i \leq B_i \\ M = \sum_{i=1}^6 [kQ_i L_i P_i + (1 - L_i)Q_i P_i] \\ T = \sum_{i=1}^6 B_i \times C_i \end{array} \right. \quad (17)$$

7.3.2 基于改进版全局粒子群算法求解模型构建

本文采用 **GBest PSO** 对优化模型进行求解。**GBest PSO** 是标准粒子群优化算法的一种变体改进 [2]，它在全局最佳位置的概念上进行了改进，以提高算法的性能。通过调整惯性权重的值，平衡粒子的全局搜索和局部搜索能力；引入了约束因子来控制粒子速度的更新，确保粒子在搜索空间中的稳定性，减少震荡，提高收敛速度。

两重改进

1. 引入先验知识，设定 30% 的初始粒子为人为设定，70% 粒子随机生成，使得粒子群算法能够更快的收敛。
2. 为了在 PSO 模型中添加约束条件，本文引入 罚函数：

$$F = \begin{cases} 20 \times (Q_i - B_i), & \text{if } Q_i > B_i \\ 0, & \text{if } Q_i \leq B_i \end{cases} \quad (15)$$

由于粒子群算法寻找最优解为函数最小值，因此需要对目标函数进行处理，修正后的目标函数为：

$$f = -(M - T) + F \quad (16)$$

7.3.3 决策模型求解结果

经过粒子群 7 轮 300 次迭代得到 7.1 至 7.7 每日收益的最大值，结果如下表：

表 18: 七日收益最大值

日期	7.1	7.2	7.3	7.4	7.5	7.6	7.7
收益	1810.02	1802.87	1745.22	1758.42	1790.44	1794.12	1799.79

商超决策结果如下所示

表 19: 7 月 1 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量（变量）	189.81	23.37	24.56	33.38	103.91	55.13
利润率（变量）	1.20	0.98	0.80	0.88	1.20	1.00

8 问题三的模型建立与求解

问题三要求在销售空间有限的约束下制定单品级别的补货与定价策略：可售单品总数控制在 27–33 个，各单品最小陈列量 2.5 千克，在尽量满足市场需求的前提下使商超

收益最大。与问题二相比，本问存在三个关键差异：（1）决策粒度从品类下沉至单品，这会导致该问计算量远超前一问，需要考虑求解前提排除一些不太可能选取的单品；（2）限制方程有所增加，包括进货单品总数目、各订购量最小值下限；（3）需要同时兼顾市场需求满足，新增的“需求度”需要提出一种量化定义，并考虑作为奖励项或是第二个优化目标。

8.1 预测销售数据及单品筛选

8.1.1 单品种类初步筛选

根据题目要求，参照 2023 年 6 月 24 日至 6 月 30 日的可售单品确定 7 月 1 日补货单品的候选范围。对该七日内出现过的单品进行整合，得到 48 个单品种类。考虑到题目要求各单品最小陈列量为 2.5 千克，本文剔除七天内单品总销量小于 2.5 kg 的单品，筛选后剩余 45 个单品进入后续预测与处理环节。

8.1.2 各单品销量及成本的预测

观察筛选出的 45 种单品的历史日销量数据，发现均存在不同程度的缺失——多数单品并非每日都有销售记录，数据的连续性远不及品类级汇总数据。采用 LSTM 等时间序列模型进行单品级预测时，缺失值过多会严重影响模型训练效果。

进一步分析发现，单品在短时间内的日销量与日成本变化无明显趋势，且无显著的日内变化规律，数据波动呈现随机性特征。因此，本文直接采用七日加权平均作为各单品在 7 月 1 日的销量与成本预测值，以最近日期数据赋予更高的权重。

对于与定价相关的销量修正，本问同样沿用问题二中的价格弹性修正函数，在品类层面对预测销量进行修正。

8.1.3 各单品需求度定义与进一步单品筛选

（1）需求度量化的逻辑

在确定哪些单品应当进货以及进货量时，需要量化消费者对各单品的“需求度”。然而，仅凭销量衡量需求是不合理的。因为部分单品的单笔销量虽然较小（如小米椒等调味类单品，每次购买量通常不足 0.5 kg），但其购买频次极高——几乎每位购买蔬菜的顾客都会顺带购买。若仅以总销量排序，这些“单笔少、频次高”的刚需单品会被低估。

因此，本文综合考虑单品销量和单品销售次数两个维度，采用多准则决策方法对需求度进行综合评价。

（2）VIKOR 法评估

本文采用多准则妥协解排序法进行需求度评价。VIKOR 方法的核心优势在于：能够同时考虑群体效用最大化和个体遗憾最小化，在多个评价准则之间寻找妥协解，具有较高的排序稳定性和可信度。

设 n 个单品在 m 个评价指标（本文 $m = 2$ ，分别为销量和销售次数）上的归一化指标矩阵为 $[n_{ij}]$ ，各指标权重为 ω_j ，则 VIKOR 方法的计算步骤如下：

步骤一：计算各单品的群体效用值 S_i （衡量与理想解的加权曼哈顿距离）：

$$S_i = \sum_{j=1}^m \omega_j \cdot \frac{\text{Max}(n_j) - n_{ij}}{\text{Max}(n_j) - \text{Min}(n_j)} \quad (18)$$

步骤二：计算各单品的个体遗憾值 R_i （衡量在最差指标上与理想解的最大差距）：

$$R_i = \max_{j=1}^m \left[\omega_j \cdot \frac{\text{Max}(n_j) - n_{ij}}{\text{Max}(n_j) - \text{Min}(n_j)} \right] \quad (19)$$

步骤三：计算折中指标值 D_i （综合群体效用和个体遗憾的妥协排序指标）：

$$D_i = \frac{\nu (S_i - S^*)}{S^- - S^*} + \frac{(1 - \nu) (R_i - R^*)}{R^- - R^*} \quad (20)$$

其中：

$$S^* = \min_{1 \leq i \leq n} S_i, \quad S^- = \max_{1 \leq i \leq n} S_i, \quad R^* = \min_{1 \leq i \leq n} R_i, \quad R^- = \max_{1 \leq i \leq n} R_i$$

（3）熵权法修正指标权重

虽然 VIKOR 理论严谨，但一般的 VIKOR 取 $\nu = 0.5$ 或主观判断两个指标重要性并赋值可能引入主观因素，为了增强结果的客观性和鲁棒性，本文引入信息熵方法自动计算权重，并代入 VIKOR 重新计算需求度。

熵值法的核心在于考察数据分布的离散程度，离散程度越高的信息量越大，权重也应越大。

步骤一：计算各准则的信息熵

对于准则 j （销量或频次），首先将标准化后的数据进行概率化处理： $p_{ij} = \frac{\tilde{f}_{ij}}{\sum_{i=1}^n \tilde{f}_{ij}}$ 计算准则 j 的信息熵：

$$E_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln(p_{ij} + \epsilon) \quad (21)$$

其中 $\epsilon = 10^{-10}$ 为数值计算中的小量，用于防止 $\ln(0)$ 的出现。

步骤二：计算客观权重

定义权重冗余度为 $d_j = 1 - E_j$ ，则各准则的客观权重为：

$$\omega_{\text{销量}}^* = \frac{1 - E_{\text{销量}}}{2 - E_{\text{销量}} - E_{\text{频次}}} \quad (22)$$

$$\omega_{\text{频次}}^* = \frac{1 - E_{\text{频次}}}{2 - E_{\text{销量}} - E_{\text{频次}}} \quad (23)$$

满足 $\omega_{\text{销量}}^* + \omega_{\text{频次}}^* = 1$ 。

步骤三：使用客观权重重新计算 **VIKOR**

将计算得到的 ω_j^* 代入 VIKOR 方法，重新计算群体效用指标 S_i^* 、个体遗憾指标 R_i^* 以及折中指标值 Q_i^* 。

(4) 求解结果

通过熵权法得到的客观权重为 $\omega_{\text{销量}}^* = 0.65$, $\omega_{\text{频次}}^* = 0.35$ 。代入 VIKOR 方法计算得到每个单品的最终需求度 D_i^{final} 。

并对 45 种候选单品 D_i^{final} 从小到大排序。为减少数据维度并降低明显不符合要求的单品对模型求解的干扰，本文根据最终需求度去除排名后 5 位的单品，最终得到 40 种单品进入后续的优化决策模型求解范围。

8.2 双目标优化模型的建立

基于更好的优化表现考虑，本文将需求度这一指标作为第二个优化目标，构建一个混合双目标优化模型，而非仅作为单目标优化模型的一奖励项。这样可以更清晰地权衡收益最大化与市场需求满足度之间的关系，得到更合理的补货与定价方案。

8.2.1 决策变量

不同于问题二以品类为单位进行补货，本问需要从 40 种候选单品中选择哪些进行补货，并确定每种单品的定价和补货量。因此，在问题二的连续决策变量基础上，还需引入 0-1 离散变量。设第 j 个单品的决策变量为：

$$\begin{aligned} \text{进货标志: } z_j, \quad (z_j \in \{0, 1\}, 1 \leq j \leq 40) \\ \text{加成率: } w_j, \quad (1 \leq j \leq 40) \\ \text{补货量: } B_j, \quad (1 \leq j \leq 40) \end{aligned} \tag{24}$$

其中 $z_j = 1$ 表示单品 j 被选中进货， $z_j = 0$ 表示不进货。

8.2.2 约束条件

约束 1 (单品总数限制): 由于销售空间有限，商超补货的可售单品总数限制在 27–33 之间：

$$27 \leq \sum_{j=1}^{40} z_j \leq 33 \tag{25}$$

约束 2 (最小陈列量与销量上限): 补货单品的最小陈列量为 2.5 kg，且总销量不能高于补货量：

$$2.5 \leq z_j \times B_j, \quad Q_j \leq B_j, \quad \forall j \tag{26}$$

约束 3 (销量修正函数): 沿用问题二的价格弹性修正函数。

8.2.3 双优化目标

目标一：最大化商超收益。商超一天的销售金额 M 和总成本 T 为：

$$\begin{aligned} M &= \sum_{j=1}^{40} [Q_j \cdot L_j \cdot P_j \cdot k + Q_j \cdot (1 - L_j) \cdot P_j] \\ T &= \sum_{j=1}^{40} B_j \times C_j \end{aligned} \quad (27)$$

利润目标为：

$$f_1 = M - T \quad (28)$$

目标二：最大化市场需求满足度。以 VIKOR 评价得到的需求度 D_j 为权重，衡量所选单品对市场需求的覆盖程度：

$$f_2 = \frac{\sum_{j=1}^{40} z_j \times D_j}{\sum_{j=1}^{40} D_j} \quad (29)$$

f_2 的取值范围为 $[0, 1]$ ，值越大表示所选单品集合对消费者需求的满足程度越高。

8.2.4 完整模型总结

综合上述要素，问题三的混合非线性双目标优化模型总结如下：

$$\max \quad f_1 = M - T \quad (30)$$

$$\max \quad f_2 = \frac{\sum_{j=1}^{40} z_j \times D_j}{\sum_{j=1}^{40} D_j} \quad (31)$$

$$\text{s.t.} \quad \left\{ \begin{aligned} &z_j \in \{0, 1\}, \quad 1 \leq j \leq 40 \\ &w_j, B_j \geq 0, \quad 1 \leq j \leq 40 \\ &27 \leq \sum_{j=1}^{40} z_j \leq 33 \\ &2.5 \leq z_j \times B_j, \quad Q_j \leq B_j \\ &\ln Q = \ln Q_{\text{预测值}} + E_p \ln \frac{P}{P_{\text{预测}}} \\ &M = \sum_{j=1}^{40} [kQ_j L_j P_j + (1 - L_j)Q_j P_j] \\ &T = \sum_{j=1}^{40} B_j \times C_j \end{aligned} \right. \quad (32)$$

8.3 基于 NSGA-II 的多目标优化求解

8.3.1 NSGA-II 算法介绍

针对本文构建的混合非线性双目标优化模型，本文选用经典且高效的 **NSGA-II** 进行求解。该算法的核心逻辑依赖于三大创新机制：首先，通过快速非支配排序将种群按 Pareto 支配关系进行逐层分级，高效筛选并保留高质量解；其次，引入拥挤度距离算子，在同一非支配层级中优先保留分布稀疏的个体，从而确保最终的 Pareto 前沿解集在目标空间内分布均匀、维持良好的种群多样性；最后，采用父代与子代合并竞争的精英保留策略，有效防止了进化迭代过程中全局最优解的丢失。

本问题求解的重心在于需平衡双目标函数，且兼具离散变量与连续变量。部分连续优化算法（如 PSO）难以直接处理混合变量结构，且单目标算法无法求得完整的 Pareto 前沿。而其他主流多目标算法中，MOEA/D 需要预设权重且对前沿形状过度敏感，SPEA2 则在处理大规模混合编码时计算开销过大。相比之下，NSGA-II 可兼容混合编码方案，且能在高度非线性的双目标问题中展现出卓越的收敛性与鲁棒性，因此成为本问题的首选算法。

8.3.2 算法改进

针对本问题的模型需求，对 NSGA-II 进行了以下改进：

（a）**BG-RI 混合编码** 标准 NSGA-II 通常采用统一的实数编码。本问的 40 个进货标志变量为 0-1 离散变量，而 80 个加成率和补货量变量为连续变量。为适应这种混合决策变量结构，本文采用 BG（Binary-Gray）编码处理 0-1 变量，采用 RI（Real Integer）编码处理连续变量，两部分编码拼接构成完整的个体染色体。交叉和变异算子分别在两个编码段上独立操作。

（b）**先验知识初始化** 与问题二类似，基于历史数据中各单品的销量排名和加成率均值，指定部分初始种群个体，使初始种群中包含高质量的候选解，加快算法收敛。

（c）**全局存档策略** 引入全局存档，在每一代迭代中将当前发现的所有非支配解存入存档，与下一代种群合并参与选择。全局存档策略能够保留历史上发现的全局最优解，避免因遗传操作的随机性导致优良个体丢失。

8.3.3 算法参数设置

经过调整，最终选取 NSGA-II 参数设置如下：

表 20: NSGA-II 算法参数设置

参数	取值	参数	取值
种群规模	100	最大迭代次数	400
编码方式	BG + RI 混合编码	早停阈值	20

8.4 Pareto 前沿解的选择

8.4.1 Pareto 前沿分析

经过 400 次迭代，NSGA-II 输出的 Pareto 前沿如图 7 所示。

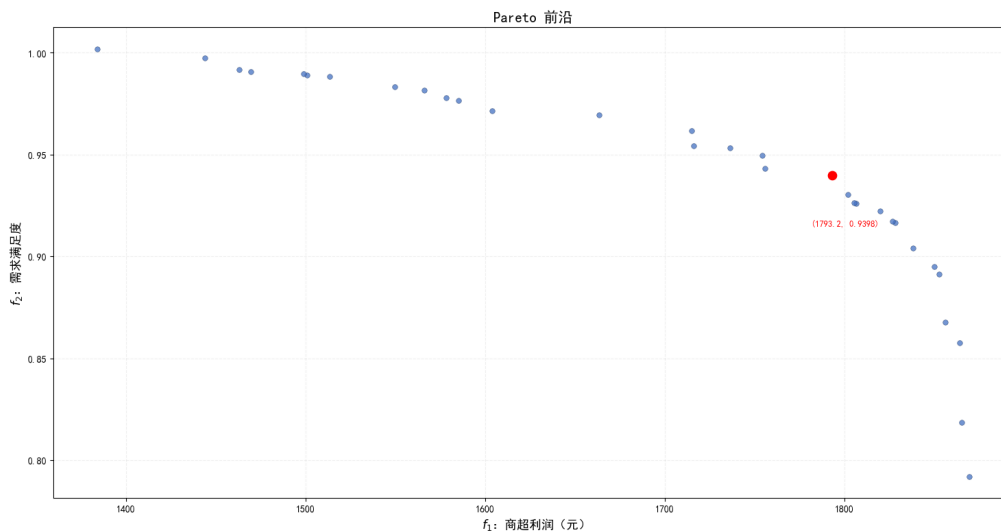


图 7: NSGA-II 求解 Pareto 前沿

观察 Pareto 前沿可以发现： f_1 与 f_2 之间存在明确的权衡关系——提高需求满足度通常需要增加低利润或高成本单品的进货量，导致整体利润下降；反之，仅选择高利润单品则会降低需求满足度。

8.4.2 最优解选择方法

Pareto 前沿上的每个解在数学意义上均为最优解（非支配解），需要根据决策偏好从中选择最终实施方案。本文考虑通过膝点法从前沿中选取最终解，其核心在于选择 Pareto 前沿上曲率最大的点，即边际权衡关系发生“拐弯”的位置。膝点代表了两个目标的最佳边际权衡，即在膝点处进一步提升任何一个目标都需要付出另一个目标上突然增大的代价。

8.4.3 最终解的确定

通过膝点法在 Pareto 前沿中确定最优解为：

$$f_1 = 1668.6 \text{ 元}, \quad f_2 = 0.9667 \quad (33)$$

此时商超选择进货的单品总数为 **30** 个，需求满足度达到 **96.67%**。

表 21: 部分单品补货量及定价策略

单品名称	分类名称	利润率	进货量/kg
净藕 (1)	水生根茎类	1.200	4.47
洪湖藕带	水生根茎类	1.194	2.72
金针菇 (盒)	食用菌	1.194	9.02
西峡花菇 (1)	食用菌	1.200	3.59
紫茄子 (2)	茄类	1.191	4.10
长线茄	茄类	0.757	6.43

部分单品的补货量及定价策略如表 21.

将本文方案的日利润 $f_1 = 1668.6$ 元与过去七天该商超的实际日均利润进行对比, 收益提升 18%。同时, 需求满足度 $f_2 = 0.9667$, 意味着所选 30 个单品覆盖了约 96.7% 的消费者需求权重, 能够满足绝大多数消费者的购买需求。

9 模型优缺点分析

9.1 模型优点

1. 在数据处理中从多个维度分析历史数据, 更好地挖掘出信息中的各规律, 包括综合两个指标得到单品分类等。同时在使用数据集前, 对异常值和无关值进行了清洗, 提高模型求解效率和决策科学性。

2. 参考了多个相关领域的研究, 引入经济学中的双对数模型等, 很大程度上提高了模型的拟合效果。

3. 在几个模型的实际求解过程中, 都通过细致研究及对比, 结合模型特征和数据规模选取最合适的算法, 并针对求解算法做了一些优化以提高求解效率。如 LSTM 的选取、GBest 的改进算法、NSGA-II 的改进算法。

9.2 模型缺点

1. 由于 LSTM 模型对历史数据的连续性和完整性存在依赖, 所以对于问题三中部分数据缺失较严重的单品, 难以直接应用时间序列模型进行较好的预测, 只能退化为加权平均估计, 从而限制了单品预测的精细度。

2. 本文在定价修正中默认价格弹性参数在预测期内保持不变, 未进一步考虑弹性系数可能随季节、价格区间、消费场景变化而动态调整的问题, 因此在更长预测期或更复杂市场环境下仍有改进空间。

参考文献

- [1] Kalyanmoy Deb, Samir Agrawal, Amrit Pratap, et al. A fast and elitist multi-objective genetic algorithm: NSGA-II[J]. *IEEE Transactions on Evolutionary Computation*, 2002, 6(2): 182–197.
- [2] Kennedy J, Eberhart R C. Particle swarm optimization[C]//Proceedings of the IEEE International Joint Conference on Neural Networks. 1995: 1942–1948.
- [3] 姚志, 何蒲明. 中国农村居民粮食消费需求及弹性测算 [J]. *统计与决策*, 2020, 36(03): 52–56.
- [4] 毛莉莎. 供应链视角下蔬菜批发市场定价策略及产销模式研究 [D/OL]. 2023.

A 程序

A.1 数据预处理与第一问

该部分代码较杂，这里给出一览表，不附具体代码。

程序名	语言	描述
combine.py	Python	将表格按时间聚合为大表
hot_one.py	Python	将分类列转化为独热编码形式
data_clean.py	Python	针对异常加成率的数据清洗
data_select.py	Python	排除无关信息的数据筛选
time.py	Python	从时间维度的销量规律研究
xiangguan.py	Python	品类间相关性与偏相关性计算
jiacheng.py	Python	加成率分布计算
FPgrowth.py	Python	FP-growth 算法计算

A.2 第二问

A.2.1 A. LSTM 成本预测代码

```
1 import pandas as pd
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 from torch.utils.data import DataLoader, TensorDataset
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import r2_score, mean_squared_error,
   mean_absolute_error
8 from math import sqrt
9 import os
10 import copy
11
12 # 设置随机种子
13 torch.manual_seed(42)
14 np.random.seed(42)
15
16 # ===== 配置参数 =====
17 categories = ['huacai', 'huaye', 'lajiao', 'qie', 'shiyongjun',
   'shuishenggen']
18 look_back = 30
19 n_future = 1
```

```

20 epochs = 400
21 batch_size = 16
22 learning_rate = 0.005
23 hidden_size = 64
24 num_layers = 1
25 dropout_rate = 0.3
26 test_days = 120
27 val_ratio = 0.2
28
29 class LSTMModel(nn.Module):
30     def __init__(self, input_size=2, hidden_size=64, num_layers=1,
31                 output_size=1, dropout=0.3):
32         super().__init__()
33         self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
34                             batch_first=True,
35                             dropout=dropout if num_layers>1 else 0)
36         self.dropout = nn.Dropout(dropout)
37         self.fc = nn.Linear(hidden_size, output_size)
38
39     def forward(self, x):
40         out, _ = self.lstm(x)
41         out = out[:, -1, :]
42         out = self.dropout(out)
43         out = self.fc(out)
44         return out
45
46 def create_multivariate_dataset(data, look_back, n_future):
47     X, Y = [], []
48     for i in range(len(data) - look_back - n_future + 1):
49         X.append(data[i:i+look_back])
50         Y.append(data[i+look_back:i+look_back+n_future, 0])
51     return np.array(X), np.array(Y)
52
53 results = []
54 future_predictions = {}
55
56 for category in categories:
57     print(f"\n{'='*50}")
58     print(f"正在处理品类: {category} 成本预测")
59     print('='*50)

```



```

60
61     file_path = f'{category}_cost.csv'
62     if not os.path.exists(file_path):
63         continue
64     df = pd.read_csv(file_path)
65     df['date'] = pd.to_datetime(df['date'])
66     df = df.sort_values('date')
67     df['weekday'] = df['date'].dt.dayofweek + 1
68
69     cost = df['cost'].values
70     weekday = df['weekday'].values
71     data = np.column_stack([cost, weekday])
72
73     train_val_data = data[:-test_days]
74     test_data = data[-test_days:]
75     val_size = int(len(train_val_data) * val_ratio)
76     train_data = train_val_data[:-val_size]
77     val_data = train_val_data[-val_size:]
78
79     scaler_cost = StandardScaler()
80     scaler_weekday = StandardScaler()
81
82     train_cost_scaled = scaler_cost.fit_transform(
83         train_data[:, 0].reshape(-1,1)).flatten()
84     train_weekday_scaled = scaler_weekday.fit_transform(
85         train_data[:, 1].reshape(-1,1)).flatten()
86     train_scaled = np.column_stack([train_cost_scaled,
87                                     train_weekday_scaled])
88
89     val_cost_scaled = scaler_cost.transform(
90         val_data[:, 0].reshape(-1,1)).flatten()
91     val_weekday_scaled = scaler_weekday.transform(
92         val_data[:, 1].reshape(-1,1)).flatten()
93     val_scaled = np.column_stack([val_cost_scaled,
94                                   val_weekday_scaled])
95
96     test_cost_scaled = scaler_cost.transform(
97         test_data[:, 0].reshape(-1,1)).flatten()
98     test_weekday_scaled = scaler_weekday.transform(
99         test_data[:, 1].reshape(-1,1)).flatten()

```

```

98     test_scaled = np.column_stack([test_cost_scaled,
99                                     test_weekday_scaled])
100
101     trainX, trainY = create_multivariate_dataset(train_scaled,
102                                                    look_back, n_future)
103     valX, valY = create_multivariate_dataset(val_scaled,
104                                                look_back, n_future)
105     testX, testY = create_multivariate_dataset(test_scaled,
106                                                  look_back, n_future)
107
108     if len(testX) == 0:
109         continue
110
111     trainX = torch.tensor(trainX, dtype=torch.float32)
112     trainY = torch.tensor(trainY, dtype=torch.float32)
113     valX = torch.tensor(valX, dtype=torch.float32)
114     valY = torch.tensor(valY, dtype=torch.float32)
115     testX = torch.tensor(testX, dtype=torch.float32)
116     testY = torch.tensor(testY, dtype=torch.float32)
117
118     train_dataset = TensorDataset(trainX, trainY)
119     train_loader = DataLoader(train_dataset,
120                               batch_size=batch_size, shuffle=True)
121
122     model = LSTMModel(input_size=2, hidden_size=hidden_size,
123                        num_layers=num_layers, output_size=n_future,
124                        dropout=dropout_rate)
125     criterion = nn.MSELoss()
126     optimizer = torch.optim.Adam(model.parameters(),
127                                   lr=learning_rate)
128
129     best_val_loss = float('inf')
130     best_model_state = None
131
132     for epoch in range(epochs):
133         model.train()
134         train_loss = 0.0
135         for batch_X, batch_Y in train_loader:
136             optimizer.zero_grad()
137             output = model(batch_X)

```

```

132         loss = criterion(output, batch_Y)
133         loss.backward()
134         optimizer.step()
135         train_loss += loss.item() * len(batch_X)
136     train_loss /= len(train_dataset)
137
138     model.eval()
139     with torch.no_grad():
140         val_pred = model(valX)
141         val_loss = criterion(val_pred, valY).item()
142
143     if val_loss < best_val_loss:
144         best_val_loss = val_loss
145         best_model_state = copy.deepcopy(model.state_dict())
146
147     if (epoch+1) % 20 == 0:
148         print(f'Epoch {epoch+1} | Train Loss: {train_loss:.6f}
149               | Val Loss: {val_loss:.6f}')
150
151     model.load_state_dict(best_model_state)
152
153     model.eval()
154     with torch.no_grad():
155         test_pred = model(testX).numpy()
156         test_true = testY.numpy()
157
158     test_pred = scaler_cost.inverse_transform(test_pred)
159     test_true = scaler_cost.inverse_transform(test_true)
160
161     pred_flat = test_pred.flatten()
162     true_flat = test_true.flatten()
163     mse = mean_squared_error(true_flat, pred_flat)
164     rmse = sqrt(mse)
165     mae = mean_absolute_error(true_flat, pred_flat)
166     r2 = r2_score(true_flat, pred_flat)
167
168     print(f"R2 = {r2:.4f}, MSE = {mse:.2f}, RMSE = {rmse:.2f}, MAE
169           = {mae:.2f}")
170
171     model.eval()

```

```

170     with torch.no_grad():
171         last_seq = test_scaled[-look_back:].copy()
172         last_weekday = test_data[-1, 1]
173         future_weekdays_raw = [(int(last_weekday) + i) % 7 + 1 for
174                                 i in range(7)]
175         future_weekdays_scaled = scaler_weekday.transform(
176             np.array(future_weekdays_raw).reshape(-1,1)).flatten()
177
178         future_preds = []
179         current_seq = last_seq.copy()
180         for day in range(7):
181             input_tensor = torch.tensor(current_seq,
182                                         dtype=torch.float32).unsqueeze(0)
183             pred_scaled = model(input_tensor).item()
184             future_preds.append(pred_scaled)
185             new_row = np.array([pred_scaled,
186                               future_weekdays_scaled[day]])
187             current_seq = np.vstack([current_seq[1:], new_row])
188
189         future_preds = scaler_cost.inverse_transform(
190             np.array(future_preds).reshape(-1,1)).flatten()
191         future_preds = np.maximum(future_preds, 0)
192         future_predictions[category] = future_preds
193
194     results.append({'品类': category, 'R2': r2, 'MSE': mse,
195                   'RMSE': rmse, 'MAE': mae})
196
197 print("\n" + "="*60)
198 print("所有品类成本预测结果汇总")
199 chinese_map = {'huacai': '花菜类', 'huaye': '花叶类', 'lajiao':
200               '辣椒类',
201               'qie': '茄类', 'shiyongjun': '食用菌',
202               'shuishenggen': '水生根茎类'}
203
204 for res in results:
205     name_cn = chinese_map.get(res['品类'], res['品类'])
206     print(f"{name_cn}\t{res['R2']:.4f}\t{res['MSE']:.2f}\t{res['RMSE']:.2f}")

```

Listing 1: LSTM 训练与成本预测

A.2.2 B. LSTM 销量预测代码

```
1 import pandas as pd
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 from torch.utils.data import DataLoader, TensorDataset
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.metrics import r2_score, mean_squared_error,
8     mean_absolute_error
9 from math import sqrt
10 import os
11 import copy
12
13 # 设置随机种子
14 torch.manual_seed(42)
15 np.random.seed(42)
16
17 categories = ['huacai', 'huaye', 'lajiao', 'qie', 'shiyongjun',
18     'shuishenggen']
19 look_back = 30
20 n_future = 1
21 epochs = 400
22 batch_size = 16
23 learning_rate = 0.005
24 hidden_size = 64
25 num_layers = 1
26 dropout_rate = 0.3
27 test_days = 120
28 val_ratio = 0.2
29
30 class LSTMModel(nn.Module):
31     def __init__(self, input_size=2, hidden_size=64, num_layers=1,
32         output_size=1, dropout=0.3):
33         super().__init__()
34         self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
35             batch_first=True,
36             dropout=dropout if num_layers>1 else 0)
37         self.dropout = nn.Dropout(dropout)
38         self.fc = nn.Linear(hidden_size, output_size)
```

```

37
38     def forward(self, x):
39         out, _ = self.lstm(x)
40         out = out[:, -1, :]
41         out = self.dropout(out)
42         out = self.fc(out)
43         return out
44
45     def create_multivariate_dataset(data, look_back, n_future):
46         X, Y = [], []
47         for i in range(len(data) - look_back - n_future + 1):
48             X.append(data[i:i+look_back])
49             Y.append(data[i+look_back:i+look_back+n_future, 0])
50         return np.array(X), np.array(Y)
51
52 results = []
53 future_predictions = {}
54
55 for category in categories:
56     print(f"\n{'='*50}")
57     print(f"正在处理品类: {category}")
58     print('='*50)
59
60     file_path = f'{category}_category.csv'
61     if not os.path.exists(file_path):
62         continue
63
64     df = pd.read_csv(file_path)
65     df['date'] = pd.to_datetime(df['date'])
66     df = df.sort_values('date')
67     df['weekday'] = df['date'].dt.dayofweek + 1
68
69     sales = df['sales'].values
70     weekday = df['weekday'].values
71     data = np.column_stack([sales, weekday])
72
73     train_val_data = data[:-test_days]
74     test_data = data[-test_days:]
75     val_size = int(len(train_val_data) * val_ratio)
76     train_data = train_val_data[:-val_size]
77     val_data = train_val_data[-val_size:]

```

```

77
78     scaler_sales = StandardScaler()
79     scaler_weekday = StandardScaler()
80
81     train_sales_scaled = scaler_sales.fit_transform(
82         train_data[:, 0].reshape(-1,1)).flatten()
83     train_weekday_scaled = scaler_weekday.fit_transform(
84         train_data[:, 1].reshape(-1,1)).flatten()
85     train_scaled = np.column_stack([train_sales_scaled,
86                                     train_weekday_scaled])
87
88     val_sales_scaled = scaler_sales.transform(
89         val_data[:, 0].reshape(-1,1)).flatten()
90     val_weekday_scaled = scaler_weekday.transform(
91         val_data[:, 1].reshape(-1,1)).flatten()
92     val_scaled = np.column_stack([val_sales_scaled,
93                                   val_weekday_scaled])
94
95     test_sales_scaled = scaler_sales.transform(
96         test_data[:, 0].reshape(-1,1)).flatten()
97     test_weekday_scaled = scaler_weekday.transform(
98         test_data[:, 1].reshape(-1,1)).flatten()
99     test_scaled = np.column_stack([test_sales_scaled,
100                                   test_weekday_scaled])
101
102     trainX, trainY = create_multivariate_dataset(train_scaled,
103                                                  look_back, n_future)
104     valX, valY = create_multivariate_dataset(val_scaled,
105                                              look_back, n_future)
106     testX, testY = create_multivariate_dataset(test_scaled,
107                                                look_back, n_future)
108
109     if len(testX) == 0:
110         continue
111
112     trainX = torch.tensor(trainX, dtype=torch.float32)
113     trainY = torch.tensor(trainY, dtype=torch.float32)
114     valX = torch.tensor(valX, dtype=torch.float32)
115     valY = torch.tensor(valY, dtype=torch.float32)
116     testX = torch.tensor(testX, dtype=torch.float32)

```

```

111 testY = torch.tensor(testY, dtype=torch.float32)
112
113 train_dataset = TensorDataset(trainX, trainY)
114 train_loader = DataLoader(train_dataset,
115                             batch_size=batch_size, shuffle=True)
116
117 model = LSTMModel(input_size=2, hidden_size=hidden_size,
118                   num_layers=num_layers, output_size=n_future,
119                   dropout=dropout_rate)
120 criterion = nn.MSELoss()
121 optimizer = torch.optim.Adam(model.parameters(),
122                               lr=learning_rate)
123
124 best_val_loss = float('inf')
125 best_model_state = None
126
127 for epoch in range(epochs):
128     model.train()
129     train_loss = 0.0
130     for batch_X, batch_Y in train_loader:
131         optimizer.zero_grad()
132         output = model(batch_X)
133         loss = criterion(output, batch_Y)
134         loss.backward()
135         optimizer.step()
136         train_loss += loss.item() * len(batch_X)
137     train_loss /= len(train_dataset)
138
139     model.eval()
140     with torch.no_grad():
141         val_pred = model(valX)
142         val_loss = criterion(val_pred, valY).item()
143
144     if val_loss < best_val_loss:
145         best_val_loss = val_loss
146         best_model_state = copy.deepcopy(model.state_dict())
147
148     if (epoch+1) % 20 == 0:
149         print(f'Epoch {epoch+1} | Train Loss: {train_loss:.6f}
150               | Val Loss: {val_loss:.6f}')

```



```

148
149     model.load_state_dict(best_model_state)
150
151     model.eval()
152     with torch.no_grad():
153         test_pred = model(testX).numpy()
154         test_true = testY.numpy()
155
156     test_pred = scaler_sales.inverse_transform(test_pred)
157     test_true = scaler_sales.inverse_transform(test_true)
158
159     pred_flat = test_pred.flatten()
160     true_flat = test_true.flatten()
161     mse = mean_squared_error(true_flat, pred_flat)
162     rmse = sqrt(mse)
163     mae = mean_absolute_error(true_flat, pred_flat)
164     r2 = r2_score(true_flat, pred_flat)
165
166     print(f"R2 = {r2:.4f}, MSE = {mse:.2f}, RMSE = {rmse:.2f}, MAE
167           = {mae:.2f}")
168
169     model.eval()
170     with torch.no_grad():
171         last_seq = test_scaled[-look_back:].copy()
172         last_weekday = test_data[-1, 1]
173         future_weekdays_raw = [(int(last_weekday) + i) % 7 + 1 for
174                                i in range(7)]
175         future_weekdays_scaled = scaler_weekday.transform(
176             np.array(future_weekdays_raw).reshape(-1,1)).flatten()
177
178         future_preds = []
179         current_seq = last_seq.copy()
180         for day in range(7):
181             input_tensor = torch.tensor(current_seq,
182                                         dtype=torch.float32).unsqueeze(0)
183             pred_scaled = model(input_tensor).item()
184             future_preds.append(pred_scaled)
185             new_row = np.array([pred_scaled,
186                               future_weekdays_scaled[day]])
187             current_seq = np.vstack([current_seq[1:], new_row])

```

```

184
185     future_preds = scaler_sales.inverse_transform(
186         np.array(future_preds).reshape(-1,1)).flatten()
187     future_preds = np.maximum(future_preds, 0)
188     future_predictions[category] = future_preds
189
190     results.append({'品类': category, 'R2': r2, 'MSE': mse,
191                   'RMSE': rmse, 'MAE': mae})
192
193 print("\n" + "="*60)
194 print("所有品类销量预测结果汇总")
195 chinese_map = {'huacai': '花菜类', 'huaye': '花叶类', 'lajiao':
196               '辣椒类',
197               'qie': '茄类', 'shiyongjun': '食用菌',
198               'shuishenggen': '水生根茎类'}
199
200 for res in results:
201     name_cn = chinese_map.get(res['品类'], res['品类'])
202     print(f"{name_cn}\t{res['R2']:.4f}\t{res['MSE']:.2f}\t{res['RMSE']:.2f}")

```

Listing 2: LSTM 训练与销量预测

A.3 第三问

A.3.1 A. VIKOR 与熵权法评估代码

```

1 import numpy as np
2 import pandas as pd
3
4 def run_vikor_entropy(file_path):
5     # 读取数据
6     df = pd.read_csv(file_path)
7     data = df[['销量', '销售次数']].values
8     n, m = data.shape
9
10    # --- 1. 熵权法计算客观权重 ---
11    # 归一化
12    norm_data = (data - data.min(axis=0)) / (data.max(axis=0) -
13                                              data.min(axis=0))
14    # 计算比重
15    p = norm_data / (norm_data.sum(axis=0) + 1e-10)

```

```

15 # 计算熵值
16 e = -1 / np.log(n) * np.sum(p * np.log(p + 1e-10), axis=0)
17 # 计算权重
18 w = (1 - e) / (np.sum(1 - e))
19
20 print(f"检测到权重: 销量={w[0]:.4f}, 销售次数={w[1]:.4f}")
21
22 # --- 2. VIKOR 计算 ---
23 f_star = norm_data.max(axis=0)
24 f_minus = norm_data.min(axis=0)
25
26 S = np.zeros(n)
27 R = np.zeros(n)
28 for i in range(n):
29     S[i] = np.sum(w * (f_star - norm_data[i, :]) /
30                  (f_star - f_minus + 1e-10))
31     R[i] = np.max(w * (f_star - norm_data[i, :]) /
32                  (f_star - f_minus + 1e-10))
33
34 S_star, S_minus = S.min(), S.max()
35 R_star, R_minus = R.min(), R.max()
36
37 v = 0.5 # 折中系数
38 Q = v * (S - S_star) / (S_minus - S_star + 1e-10) + \
39     (1 - v) * (R - R_star) / (R_minus - R_star + 1e-10)
40
41 # 需求度 D = 1 - Q (Q越小越好, D越大越好)
42 df['需求度_D'] = 1 - Q
43
44 # 排序并取前 40
45 result = df.sort_values(by='需求度_D',
46                        ascending=False).head(40)
47 result.to_csv("top40_items_result.csv", index=False)
48 print("需求度评估完成, 结果已保存至 top40_items_result.csv")
49 return result
50
51 # 使用示例
52 # df_top40 = run_vikor_entropy('input_vikor.csv')

```

Listing 3: 需求度评估 (VIKOR 与熵权法)

A.3.2 B. NSGA-II 双目标优化求解代码

```
1 import numpy as np
2 import pandas as pd
3 import geatpy as ea
4 import matplotlib.pyplot as plt
5
6 class ReplenishmentProblem(ea.Problem):
7     def __init__(self, item_data):
8         self.item_data = item_data
9         name = 'Replenishment_Optimization'
10        M = 2 # 目标数: 1.总利润, 2.总需求度满足
11        maxormins = [-1, -1] # -1 表示最大化
12        Dim = 120 # 40个z(0/1), 40个w(加成率), 40个B(补货量)
13
14        # 变量初始化
15        lb = [0]*40 + [0.1]*40 + [0.0]*40 # 最小加成率设为10%
16        ub = [1]*40 + [2.0]*40 + [100.0]*40 # 最大加成率200%,
17        # 补货100kg
18        varTypes = [1]*40 + [0]*40 + [0]*40 # 整数, 连续, 连续
19
20        super().__init__(name, M, maxormins, Dim, varTypes, lb, ub)
21
22    def evalVars(self, Vars):
23        # 拆分变量
24        z = Vars[:, 0:40] # 是否进货
25        w = Vars[:, 40:80] # 利润率
26        B = Vars[:, 80:120] # 进货量
27
28        # 获取输入参数
29        cost = self.item_data['成本'].values
30        q_pred = self.item_data['预测销量'].values
31        loss = self.item_data['损耗率'].values
32        demand_d = self.item_data['需求度_D'].values
33        elasticity = self.item_data['价格弹性'].values
34
35        N = Vars.shape[0]
36        f1 = np.zeros(N) # 利润
37        f2 = np.zeros(N) # 需求度
38        CV = np.zeros((N, 4)) # 约束
```

```

38
39     k = 0.7 # 打折率
40
41     for i in range(N):
42         # 价格弹性修正后的实际销量 Q
43         #  $Q = q_{pred} * (1 + w)^{elasticity}$ 
44         q_actual = q_pred * np.power((1 + w[i]), elasticity) *
45             z[i]
46
47         # 目标 1: 利润 (销售额 - 总成本)
48         revenue = np.sum(q_actual * (1-loss) * cost * (1+w[i])
49             +
50             q_actual * loss * cost * (1+w[i]) * k)
51         total_cost = np.sum(z[i] * B[i] * cost)
52         f1[i] = revenue - total_cost
53
54         # 目标 2: 需求满足度
55         f2[i] = np.sum(z[i] * demand_d)
56
57         # 约束条件 (<= 0 则满足)
58         # 1. 进货品种数在 27-33 之间
59         total_types = np.sum(z[i])
60         CV[i, 0] = 27 - total_types
61         CV[i, 1] = total_types - 33
62
63         # 2. 最小陈列量约束:  $z_i=1$  时,  $B_i \geq 2.5$ 
64         CV[i, 2] = np.sum(np.maximum(0, 2.5 * z[i] - B[i]))
65
66         # 3. 销量不能超过进货量:  $q_{actual} \leq B_i$ 
67         CV[i, 3] = np.sum(np.maximum(0, q_actual - B[i]))
68
69     return np.column_stack([f1, f2]), CV
70
71 def solve_multi_objective(csv_path):
72     # 载入数据
73     data = pd.read_csv(csv_path)
74
75     # 实例化问题
76     problem = ReplenishmentProblem(data)

```

```

76 # 构建算法 (NSGA-II)
77 algorithm = ea.moea_NSGA2_templet(
78     problem,
79     ea.Population(Encoding='RI', NIND=100),
80     MAXGEN=500, # 迭代500代
81     logTras=100
82 )
83
84 # 求解
85 res = ea.optimize(algorithm, verbose=True, drawing=1,
86                   outputMsg=True, drawLog=False)
87
88 # 绘制 Pareto 前沿
89 if res['ObjV'] is not None:
90     plt.figure(figsize=(8, 5))
91     plt.scatter(res['ObjV'][:, 0], res['ObjV'][:, 1], c='red')
92     plt.xlabel('Total Profit (Yuan)')
93     plt.ylabel('Demand Satisfaction Score')
94     plt.title('Pareto Front - Vegetable Replenishment')
95     plt.grid(True)
96     plt.show()
97
98 return res

```

Listing 4: 改进 NSGA-II 多目标优化求解

B 数据表格

B.1 第二问

7 日决策结果

表 22: 7 月 1 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量 (变量)	189.81	23.37	24.56	33.38	103.91	55.13
利润率 (变量)	1.20	0.98	0.80	0.88	1.20	1.00

表 23: 7 月 2 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量（变量）	186.08	24.03	25.50	34.46	106.21	56.37
利润率（变量）	1.20	0.97	0.80	0.85	1.20	0.99

表 24: 7 月 3 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量（变量）	185.61	24.35	22.16	43.30	103.70	57.76
利润率（变量）	1.20	0.97	0.80	0.77	1.20	1.00

表 25: 7 月 4 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量（变量）	186.78	24.31	25.09	45.03	103.77	55.78
利润率（变量）	1.20	0.94	0.78	0.89	1.20	0.99

表 26: 7 月 5 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量（变量）	185.97	25.97	24.77	32.30	103.44	55.75
利润率（变量）	1.20	0.98	0.78	0.87	1.20	0.98

表 27: 7 月 6 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量（变量）	184.73	23.97	25.24	32.81	105.30	61.69
利润率（变量）	1.20	0.97	0.79	0.90	1.20	0.98

表 28: 7 月 7 日定价与补货策略

指标	花叶类	花菜类	根茎类	茄类	辣椒类	食用菌
进货量（变量）	187.51	24.31	24.81	35.24	103.97	58.03
利润率（变量）	1.20	0.97	0.80	0.81	1.20	1.00